SUP DE VINCI · MASTÈRE CYBERSÉCURITÉ

Vigenère-Squad

Cryptanalyse de chiffrements classiques par un système de quatre agents LLM encadrés

Cryptographie appliquée et IA agentique

BENNAT Billal

Enseignant : Yulliwas AMEUR

Dépôt du projet
https://github.com/Billal93/vigenere-squad

Résumé. Ce rapport présente Vigenère-Squad, un système qui casse automatiquement six chiffrements classiques (César, affine, Vigenère, Beaufort, autoclave et substitution simple) au moyen de quatre agents spécialisés pilotés par un orchestrateur explicite. Les opérations cryptographiques et statistiques sont confiées à des outils déterministes ; les modèles de langage, de petite taille et exécutés localement sur processeur, choisissent les outils dans des listes fermées et rédigent des explications, mais ne décident jamais seuls du succès d'une attaque. L'évaluation porte sur 180 chiffrés à graine fixe pour la référence sans modèle, et sur un échantillon volontairement minuscule pour les deux modèles testés (trois chiffrés avec trois répétitions pour le plus petit, un chiffré pour l'autre), parce que la machine n'a pas de carte graphique. Sans modèle, le système retrouve le clair exact pour 83,3 % des chiffrés de la variante de base. Le rapport décrit aussi un modèle de menace du système lui-même, avec des contre-mesures dont l'effet est mesuré, et une ablation qui compare l'apport réel des agents à celui des outils seuls. Tous les nombres de ce document sont lus dans `results/metrics.json` au moment de la construction du PDF.

Mots-clés : cryptanalyse classique, indice de coïncidence, agents LLM, reproductibilité, ablation, sécurité des agents.

Abstract. This report presents Vigenère-Squad, a system that automatically breaks six classical ciphers (Caesar, affine, Vigenère, Beaufort, autokey and simple substitution) with four specialised agents driven by an explicit orchestrator. Cryptographic and statistical operations are delegated to deterministic tools; the small, locally-run language models only pick tools from closed lists and write explanations, and never alone decide whether an attack succeeded. The reference without any model is evaluated on 180 fixed-seed ciphertexts and recovers the exact plaintext for 83,3 % of the base variant. Two models are evaluated on a deliberately tiny sample (three ciphertexts with three repetitions for the smaller one, one ciphertext for the other) because the machine has no GPU. The report also provides a threat model of the system itself, with measured countermeasures, and an ablation comparing the real contribution of agents to that of the tools alone. Every number in this document is read from `results/metrics.json` when the PDF is built.

- 1. Introduction
- 2. Cadre : procédés et outils de cryptanalyse
  - 2.1 Les six procédés
  - 2.2 Reconnaître la famille : indice de coïncidence
  - 2.3 Longueur de clé : Kasiski, Friedman, indice par colonne
  - 2.4 Retrouver la clé
  - 2.5 Juger un clair : modèle de quadrigrammes
  - 2.6 Distance d'unicité
- 3. Architecture du système
  - 3.1 Vue d'ensemble
  - 3.2 Les quatre agents
  - 3.3 Ce que le modèle ne fait pas
  - 3.4 Robustesse aux défaillances
  - 3.5 Traçabilité

    - 3.6 Le Pédagogue et les exercices
- 4. Méthode d'évaluation
    - 4.1 Corpus
    - 4.2 Mesures
    - 4.3 Trois configurations pour l'ablation
    - 4.4 Le protocole réduit pour les modèles
    - 4.5 Intégrité de l'évaluation
- 5. Résultats
    - 5.1 La référence sans modèle
    - 5.2 Robustesse aux variantes
    - 5.3 Kasiski face à l'autoclave
    - 5.4 Distance d'unicité
    - 5.5 Ablation : que font les modèles ?
- 6. Modèle de menace du système lui-même
    - 6.1 Données d'outil empoisonnées
    - 6.2 Outil halluciné
    - 6.3 Consigne cachée dans le clair
    - 6.4 Résultat de l'expérience d'injection
- 7. Discussion et limites
    - 7.1 Ce que les agents apportent réellement
    - 7.2 Limites
    - 7.3 Suite possible
- 8. Reproductibilité
- 9. Conclusion
- Annexe A. Intégrité et usage de l'IA

1. Introduction

Les chiffrements classiques ont disparu de la pratique, mais ils restent un excellent terrain pour étudier ce que valent des agents fondés sur des modèles de langage lorsqu'on les applique à la cryptographie. Le problème y est bien posé : un texte chiffré, un ensemble connu de procédés, un critère de réussite objectif (retrouver le clair), et des outils statistiques éprouvés depuis plus d'un siècle. On peut donc vérifier chaque affirmation du système par un calcul, ce qui est précisément ce que l'on veut exiger d'un agent dans un domaine où une réponse plausible mais fausse a des conséquences.

La question posée par le cours est de savoir ce qu'apporte réellement une architecture multi-agents par rapport à un programme classique. Le présent projet y répond de façon empirique. Il construit d'abord un système complet et reproductible, ensuite il le fait tourner dans trois configurations (sans aucun modèle, avec un très petit modèle, avec un modèle de taille différente), enfin il compare les résultats. Il s'intéresse aussi à la sécurité du système lui-même, c'est-à-dire à ce qu'un texte déchiffré malveillant, un outil défaillant ou un modèle qui invente un appel d'outil peuvent provoquer.

Le travail est volontairement encadré par des règles d'intégrité. Aucune valeur chiffrée ne figure dans ce rapport sans provenir d'une exécution réelle consignée dans `results/metrics.json` ; les résultats défavorables sont publiés au même titre que les autres ; aucune opération cryptographique n'est déléguée à un modèle ; aucun secret n'est stocké dans le dépôt, les invites ou les traces. L'assistant de programmation Claude Code a écrit le code et les premières versions des textes à partir de mes instructions, ce que précise le fichier `AI_USAGE.md`.

Le plan est le suivant. La section 2 rappelle les procédés et les outils statistiques employés. La section 3 décrit l'architecture. La section 4 présente la méthode d'évaluation. La section 5 rapporte les résultats, la section 6 le modèle de menace et ses mesures, la section 7 discute de l'apport des agents et des limites. La section 8 détaille la reproductibilité, et la section 9 conclut.

2. Cadre : procédés et outils de cryptanalyse

2.1 Les six procédés

Tous les procédés opèrent sur l'alphabet latin de 26 lettres, après normalisation du texte (majuscules, suppression des accents, des espaces et de la ponctuation). Le chiffre de César décale chaque lettre d'un même nombre de positions. Le chiffre affine applique la transformation $c = a \cdot p + b$ modulo 26, avec a premier avec 26, ce qui laisse 312 clés. Le chiffre de Vigenère décale chaque lettre d'une valeur qui dépend de sa position modulo la longueur de la clé ; le chiffre de Beaufort

est sa variante réciproque ($c = k - p$). L'autoclave utilise comme clé une amorce suivie du texte clair lui-même, de sorte que la clé ne se répète pas. La substitution simple remplace chaque lettre par une autre selon une permutation de l'alphabet, soit 26! clés.

2.2 Reconnaître la famille : indice de coïncidence

L'indice de coïncidence d'un texte est la probabilité que deux lettres tirées au hasard soient identiques. Il vaut environ 0,0746 pour le français, 0,0667 pour l'anglais et $1/26 \approx 0{,}0385$ pour un texte uniformément aléatoire. Les procédés monoalphabétiques (César, affine, substitution) préservent l'indice du clair, alors que les procédés polyalphabétiques le rapprochent de la valeur aléatoire, d'autant plus que la clé est longue. Cette seule mesure permet déjà au Classifieur de séparer deux familles.

2.3 Longueur de clé : Kasiski, Friedman, indice par colonne

Pour un chiffre polyalphabétique à clé répétée, la méthode de Kasiski repère des séquences répétées dans le chiffré et regarde les écarts entre leurs occurrences : la longueur de clé divise souvent ces écarts. Le test de Friedman estime la longueur à partir de l'indice de coïncidence global. Le système utilise surtout une méthode par colonnes : pour chaque longueur candidate L, on découpe le texte en L colonnes et on calcule l'indice moyen de colonne ; on retient la plus petite longueur dont l'indice atteint 90 % du meilleur. Si l'indice global dépasse 85 % de celui de la langue, la longueur 1 est retenue (cas monoalphabétique).

2.4 Retrouver la clé

Pour César, un test du χ^2 entre les fréquences observées et celles de la langue donne le décalage. Pour l'affine, une recherche exhaustive des 312 clés est notée par un modèle de quadrigrammes. Pour Vigenère et Beaufort, une fois la longueur connue, chaque colonne est un César (ou son opposé), résolu par χ^2. Pour l'autoclave, Kasiski ne s'applique pas puisque la clé ne se répète pas : une montée de colline sur les lettres de l'amorce, guidée par le score de quadrigrammes du texte entier, la retrouve. Pour la substitution, un recuit simulé explore l'espace des permutations, noté par le même score.

2.5 Juger un clair : modèle de quadrigrammes

Le Linguiste s'appuie sur un modèle de langue par quadrigrammes de lettres, appris sur des livres de Project Gutenberg, avec un plancher de probabilité de 0,01 occurrence pour les quadrigrammes jamais vus. Le score d'un texte est le logarithme décimal moyen de la probabilité d'un quadrigramme. Les livres qui servent à apprendre le modèle sont disjoints de ceux dont on tire les clairs de test. Un seuil de plausibilité de $-5{,}5$ a été calibré sur ces données : les clairs réels obtiennent au moins $-4{,}6$ et les mauvaises hypothèses au plus $-7{,}6$, ce qui laisse une marge confortable de part et d'autre.

2.6 Distance d'unicité

Shannon définit la distance d'unicité d'un chiffrement comme la longueur de texte à partir de laquelle une seule clé reste compatible avec un clair de la langue : U = H(K) / D, où H(K) est l'entropie de la clé et D la redondance du clair par lettre. Pour une clé de Vigenère de longueur k, $H(K) = k \cdot \log_2 26$. Cette quantité sert de borne de référence à la section 5.4, où l'on confronte la longueur minimale réellement nécessaire à l'attaque à cette valeur théorique.

3. Architecture du système

3.1 Vue d'ensemble

Le système est un programme Python dont l'orchestrateur est écrit explicitement : il ne laisse pas un modèle décider de l'ordre d'exécution. Il maintient un état partagé typé (modèles Pydantic) qui contient le chiffré normalisé, la famille supposée, les procédés déjà écartés, les tentatives et le statut. À chaque itération, il consulte successivement le Classifieur, le Cryptanalyste et le Linguiste. Si le Linguiste juge le clair plausible, la boucle s'arrête et le Pédagogue explique la démarche. Sinon, le procédé est écarté et le Classifieur est reconsulté, dans la limite de quatre itérations.

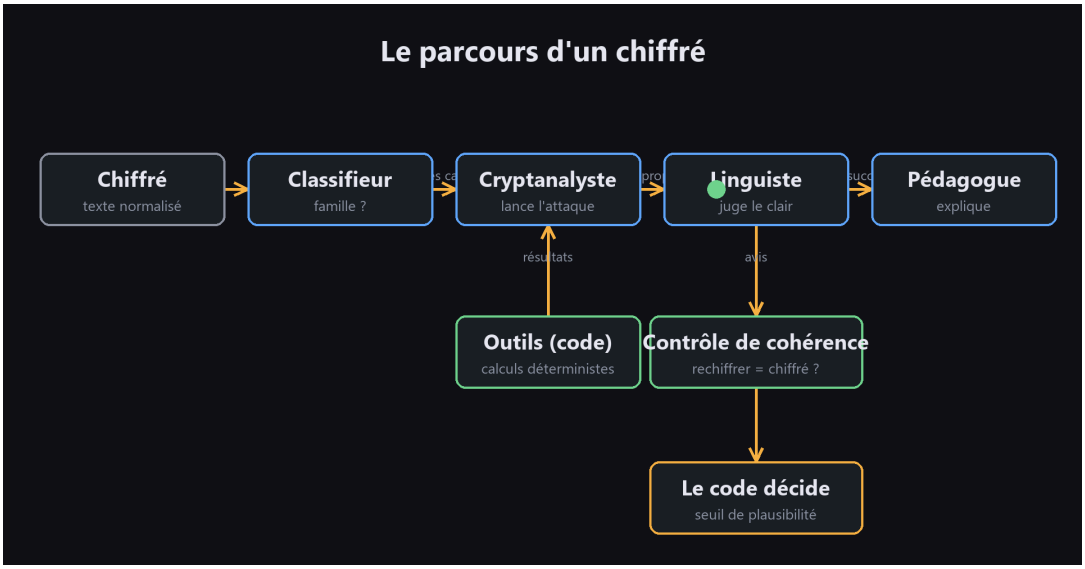


Figure : le parcours d'un chiffré. Les éléments verts sont du code déterministe.

3.2 Les quatre agents

Chaque agent possède un prompt système versionné (fichier `prompts/<agent>.v1.md`), une liste fermée d'outils et un schéma de sortie JSON validé par Pydantic. Le tableau suivant résume leurs rôles.

Agent	Rôle	Outils autorisés
Classifieur	proposer la famille et les procédés candidats	indice de coïncidence, fréquences
Cryptanalyste	choisir l'attaque et ses paramètres	Kasiski, Friedman, longueur par indice, χ^2 de César, force brute affine, attaques Vigenère, Beaufort, autoclave, substitution
Linguiste	juger la plausibilité du clair	score de quadrigrammes
Pédagogue	expliquer la démarche à un étudiant	aucun

3.3 Ce que le modèle ne fait pas

Trois dispositions limitent le rôle du modèle. D'abord, le texte chiffré n'est jamais transmis par les arguments que le modèle écrit : l'exécuteur d'outils l'injecte depuis le contexte, le modèle ne fournit que des paramètres comme une longueur de clé. Ensuite, la décision de plausibilité appartient au code : le score de quadrigrammes est calculé par l'outil et comparé au seuil, l'avis du Linguiste étant seulement journalisé s'il diverge. Enfin, un contrôle de cohérence rechiffre le clair proposé avec la clé annoncée et exige de retrouver exactement le chiffré, ce qui écarte un résultat d'outil falsifié même si le texte proposé est du français parfait.

Le contrôle de cohérence est du code : il rechiffre le clair annoncé avec la clé annoncée.

outils/coherence.py : contrôle de cohérence (lignes 13 à 36)

```
13      """Rechiffre `clair` (lettres) avec la clé telle que l'outil de cryptanalyse l'annonce ; None si clé illisible."""
14      if cle is None:
15          return None
16      try:
17          if procede == "cesar":
18              return ch.cesar_chiffrer(clair, int(cle))
19          if procede == "affine":
20              a, b = (int(x) for x in cle.split(","))
21              return ch.affine_chiffrer(clair, a, b)
22          if procede == "vigenere":
23              return ch.vigenere_chiffrer(clair, cle)
24          if procede == "beaufort":
25              return ch.beaufort(clair, cle)
26          if procede == "autoclave":
27              return ch.autoclave_chiffrer(clair, cle)
28          if procede == "substitution":
29              return ch.substitution_chiffrer(clair, cle)
30      except (ValueError, KeyError, IndexError):
31          return None
32      return None
33
34
35  def coherent(procede: str, cle: str | None, clair: str, chiffre: str) -> bool:
36      return rechiffrer(procede, cle, ch.normaliser(clair)) == chiffre
```

L'orchestrateur applique ce contrôle avant d'accepter une hypothèse ; l'avis du modèle n'est que journalisé.

orchestrateur.py : le code fait foi (lignes 103 à 118)

```
        resultat = cr.executeur.appeler(appel.outil, appel.arguments)
        procede = _procede(appel.outil)

        obs_li = li.observer(etat, resultat["clair"])
        if self.masquer_extraits:
            obs_li["extrait"] = "(non fourni)"
        sortie_li, source = li.decider(etat, obs_li)
        plausible = obs_li["score"]["plausible"]       # le code fait foi ; l'avis du modèle est journalisé
        if plausible and self.verifier_coherence and not coherent(procede, resultat["cle"], resultat["clair"], et
            journal.ecrire("incoherence_outil", "orchestrateur", procede=procede, cle=resultat["cle"])
            plausible = False
        if source == "llm" and sortie_li.plausible != plausible:
            journal.ecrire("desaccord_linguiste", "linguiste", avis_modele=sortie_li.plausible, score=plausible)
        etat.tentatives.append(Tentative(procede=procede, outil=appel.outil, arguments=appel.arguments,
                                         cle=resultat["cle"], score_moyen=resultat["score_moyen"],
                                         plausible=plausible))
```

3.4 Robustesse aux défaillances

Quand le modèle renvoie un JSON invalide, une réponse vide ou un objet qui ne respecte pas le schéma, l'agent réessaie une fois puis se replie sur sa décision déterministe, ce que la trace mentionne. Un appel à un outil hors de la liste de l'agent est refusé avant exécution, journalisé sous le type `outil_refuse`, et traité comme une réponse invalide. Un outil en panne ou trop lent déclenche une reprise puis un abandon propre. Ces comportements sont couverts par des tests d'injection de pannes.

agents/base.py : liste fermée d'outils (lignes 28 à 40)

```
class Executeur:
    def __init__(self, agent: str, ctx: ContexteOutil, journal: Journal, delai_s: float = 120.0,
                 pannes: Callable[[str], None] | None = None):
        self.agent, self.ctx, self.journal, self.delai_s, self.pannes = agent, ctx, journal, delai_s, pannes
        self.autorises = OUTILS_PAR_AGENT[agent]

    def verifier_autorise(self, outil: str) -> None:
        if outil not in self.autorises:
            self.journal.ecrire("outil_refuse", self.agent, outil=outil, autorises=sorted(self.autorises))
            raise OutilRefuse(f"{self.agent} ne peut pas appeler « {outil} »")

    def appeler(self, outil: str, arguments: dict[str, Any] | None = None) -> dict[str, Any]:
        arguments = dict(arguments or {})
```

3.5 Traçabilité

Chaque message envoyé au modèle, chaque réponse, chaque appel d'outil, chaque refus et chaque décision est écrit dans un fichier JSONL horodaté, un par chiffré. La commande `vsquad rejouer` relit une trace, et avec l'option `--verifier` contrôle que chaque appel d'outil figure bien dans la liste de son agent.

Une résolution sans modèle d'un chiffré de César, puis le rejeu de la trace avec vérification des listes d'outils.

```
$ vsquad resoudre "KHOOR ZRUOG DYHF OH PRQGH HQWLHU SRXU SDVVHU OD PXLW ODVV" --modele aucun
09:19:48.240 orchestrateur debut               {"longueur": 47, "langue": "fr", "modele": "sans_llm"}
09:19:48.257 classifieur   outil_appel        {"outil": "indice_coincidence", "arguments": {}}
09:19:48.399 classifieur   outil_resultat     {"outil": "indice_coincidence", "resultat": {"ic": 0.062, "ic_langue":
   0.0694, "ic_aleatoire": 0.0385, "longueur": 47}}
09:19:48.410 classifieur   outil_appel        {"outil": "frequences", "arguments": {}}
09:19:48.437 classifieur   outil_resultat     {"outil": "frequences", "resultat": {"plus_frequentes": {"H": 0.149, "
  O": 0.128, "D": 0.085, "R": 0.085, "U": 0.085, "V": 0.085}}}
09:19:48.444 classifieur   decision           {"source": "sans_llm", "sortie": {"famille": "monoalphabetique", "proc
  edes": ["cesar", "affine", "substitution"], "justification": "IC comparé à 85 % de celui de la langu
09:19:48.448 cryptanalyste decision           {"source": "sans_llm", "sortie": {"appel": {"outil": "chi2_cesar", "ar
  guments": {}}, "justification": "Premier procédé non écarté : cesar."}}
09:19:48.450 cryptanalyste outil_appel        {"outil": "chi2_cesar", "arguments": {}}
09:19:48.483 cryptanalyste outil_resultat     {"outil": "chi2_cesar", "resultat": {"cle": "3", "clair": "HELLOWORLDA
  VECLEMONDEENTIERPOURPASSERLAMUITLASS", "score_moyen": -4.776, "plausible": true}}
09:19:48.493 linguiste     outil_appel        {"outil": "score_quadrigrammes", "arguments": {"texte": "HELLOWORLDAVE
  CLEMONDEENTIERPOURPASSERLAMUITLASS"}}
09:19:48.522 linguiste     outil_resultat     {"outil": "score_quadrigrammes", "resultat": {"score_moyen": -4.776, "
  seuil": -5.5, "plausible": true}}
09:19:48.524 linguiste     decision           {"source": "sans_llm", "sortie": {"plausible": true, "justification":
  "Score de quadrigrammes comparé au seuil."}}
09:19:48.527 pedagogue     decision           {"source": "sans_llm", "sortie": {"explication": "Le texte a été chiff
  ré avec le procédé « cesar ». L'analyse des fréquences et de l'indice de coïncidence a permis de ret
09:19:48.529 orchestrateur fin                {"statut": "succes", "motif": null, "procede": "cesar", "iterations":
  1}

statut=succes procédé=cesar clé=3 itérations=1
clair : HELLO WORLD AVEC LE MONDE ENTIER POUR PASSER LA MUIT LASS
explication : Le texte a été chiffré avec le procédé « cesar ». L'analyse des fréquences et de l'indice de coïncidence
   a permis de retrouver la clé « 3 », ce qui donne un texte français lisible.
```

```
$ vsquad rejouer traces/exemple/manuel.jsonl --verifier
09:10:50.880 orchestrateur debut              {"longueur": 47, "langue": "fr", "modele": "sans_llm"}
09:10:50.880 classifieur   outil_appel        {"outil": "indice_coincidence", "arguments": {}}
09:10:50.894 classifieur   outil_resultat     {"outil": "indice_coincidence", "resultat": {"ic": 0.062, "ic_langue":
   0.0694, "ic_aleatoire": 0.0385, "longueur": 47}}
09:10:50.894 classifieur   outil_appel        {"outil": "frequences", "arguments": {}}
09:10:50.896 classifieur   outil_resultat     {"outil": "frequences", "resultat": {"plus_frequentes": {"H": 0.149, "
  O": 0.128, "D": 0.085, "R": 0.085, "U": 0.085, "V": 0.085}}}
09:10:50.898 classifieur   decision           {"source": "sans_llm", "sortie": {"famille": "monoalphabetique", "proc
  edes": ["cesar", "affine", "substitution"], "justification": "IC comparé à 85 % de celui de la langu
09:10:50.898 cryptanalyste decision           {"source": "sans_llm", "sortie": {"appel": {"outil": "chi2_cesar", "ar
  guments": {}}, "justification": "Premier procédé non écarté : cesar."}}
09:10:50.900 cryptanalyste outil_appel        {"outil": "chi2_cesar", "arguments": {}}
09:10:50.902 cryptanalyste outil_resultat     {"outil": "chi2_cesar", "resultat": {"cle": "3", "clair": "HELLOWORLDA
  VECLEMONDEENTIERPOURPASSERLAMUITLASS", "score_moyen": -4.776, "plausible": true}}
09:10:50.904 linguiste     outil_appel        {"outil": "score_quadrigrammes", "arguments": {"texte": "HELLOWORLDAVE
  CLEMONDEENTIERPOURPASSERLAMUITLASS"}}
09:10:50.904 linguiste     outil_resultat     {"outil": "score_quadrigrammes", "resultat": {"score_moyen": -4.776, "
  seuil": -5.5, "plausible": true}}
09:10:50.906 linguiste     decision           {"source": "sans_llm", "sortie": {"plausible": true, "justification":
  "Score de quadrigrammes comparé au seuil."}}
09:10:50.907 pedagogue     decision           {"source": "sans_llm", "sortie": {"explication": "Le texte a été chiff
  ré avec le procédé « cesar ». L'analyse des fréquences et de l'indice de coïncidence a permis de ret
09:10:50.908 orchestrateur fin                {"statut": "succes", "motif": null, "procede": "cesar", "iterations":
  1}
09:19:48.240 orchestrateur debut              {"longueur": 47, "langue": "fr", "modele": "sans_llm"}
09:19:48.257 classifieur   outil_appel        {"outil": "indice_coincidence", "arguments": {}}
09:19:48.399 classifieur   outil_resultat     {"outil": "indice_coincidence", "resultat": {"ic": 0.062, "ic_langue":
   0.0694, "ic_aleatoire": 0.0385, "longueur": 47}}
09:19:48.410 classifieur   outil_appel        {"outil": "frequences", "arguments": {}}
[… 21 lignes non affichées]
```

3.6 Le Pédagogue et les exercices

Au-delà de l'explication, le Pédagogue sait générer des exercices. À partir d'un clair du corpus, d'un procédé et d'une graine, le système tire une clé, chiffre le texte, puis exécute l'orchestrateur sans modèle pour produire le corrigé. L'exercice n'est publié que si le clair retrouvé est identique au clair d'origine ; sinon il est rejeté et un autre est tiré. Une seconde fonction, indépendante, revérifie tout exercice publié en rechiffrant le clair du corrigé avec la clé du corrigé et en recalculant les valeurs d'indice de coïncidence qui y figurent.

```
$ vsquad exercice --procede vigenere --graine 4
Exercice (vigenere, 300 lettres) : Le texte a été chiffré avec une clé répétée. Retrouvez la longueur de la clé, puis
  la clé.

GBHPRVNKLYBISWHGMSSTGMBZPMFGMXCJIVTRMFGSORHBZBEXKHSMYIVGSXDSSPPHPUIYPGPXUBIUROREYIPCLMMHPGVHJBHHGTHIUGIOCOBFIVGEHPFEIW
  FHZTDWGCPBENIWZTRQSIRCBBDSINCYFQAIULRRDHLGWUBGMUWGMBHXIPUGBBPWUGUAMCWVGVYDPWVWVXUHUWKOPYTGQWWBDDRVFDSMCXCIHZTPTKVUBJ
  MMFKDIAVYGCYBOJROCLQDTENCVLGGJTCLIFNMUVKBOPYUGQLFLIHQUPGRLCUZBMGIO

--- Corrigé (vérifié par les outils : oui) ---
{
 "cle": {
  "cle": "DXMPECC"
 },
 "clair": "DEVANTLHOMMEQUEJADOREJENAIDEJAQUETROPTROMPEENMAVIEVAJETEPARDONNESITUNEMAIMESPLUSJENAIPASLETEMPSDERELIREMALE
  TTRECESTPEUDECHOSEAMESYEUXQUEDEPAYERDELAVIELESJOURSHEUREUXQUEJEVIENSDEPASSERDANSTESBRASTUSAISQUILSMECOUTERONTDAVANTA
  GECHAPITREXXIDIALOGUEAVECUNMAITREALASOURFRAILTYISTHECAUSENOTWEFORSUCHASWEAREM",
 "etapes": {
  "ic_du_chiffre": 0.0439,
  "ic_du_clair": 0.0746,
  "tentatives": [
   [
    "vigenere",
    true
   ]
  ]
 },
 "explication": "Le texte a été chiffré avec le procédé « vigenere ». L'analyse des fréquences et de l'indice de coïnc
  idence a permis de retrouver la clé « DXMPECC », ce qui donne un texte français lisible."
}
```

4. Méthode d'évaluation

4.1 Corpus

Le corpus est généré à graine fixe (42) à partir d'extraits de livres de Project Gutenberg, dont les empreintes SHA-256 sont vérifiées. Pour chacun des six procédés et chacune des six longueurs de texte (80, 150, 300, 600, 1000 et 2000 lettres), cinq chiffrés sont produits, soit 180 chiffrés par variante. Quatre variantes sont évaluées : la base en français, une variante sans espaces ni ponctuation, une variante avec 5 % de fautes de frappe dans le chiffré, et une variante en anglais. Le système normalise le texte avant toute analyse ; on s'attend donc à ce que la variante sans espaces donne exactement les mêmes résultats que la base, ce que l'on vérifie.

4.2 Mesures

Pour chaque chiffré on enregistre le clair obtenu, le procédé reconnu, la clé, le nombre d'itérations, le temps écoulé et le pic de mémoire Python. Les mesures publiées sont : le pourcentage de clairs exacts par procédé et par tranche de longueur ; la plus petite longueur à partir de laquelle toutes les tranches atteignent 90 % de succès ; le nombre moyen d'itérations ; le temps et la mémoire par chiffré ; la proportion de chiffrés dont la famille est correctement reconnue ; enfin, pour les explications, une grille de dix critères binaires appliquée à dix explications tirées au hasard.

4.3 Trois configurations pour l'ablation

La configuration (a) n'utilise aucun modèle : l'orchestrateur est le même, mais chaque agent prend sa décision déterministe. La configuration (b) utilise `qwen3:0.6b`, très petit modèle. La configuration (c) utilise `qwen3:4b`, modèle de taille différente. Les deux modèles tournent via Ollama 0.40.13, sur

processeur uniquement, avec une température de 0,3, un contexte de 8192 jetons, des graines 43, 44 et 45 pour les trois répétitions, et une sortie contrainte par schéma JSON. Leurs empreintes (digests) sont consignées dans `config/experiment.yaml` et vérifiées par `scripts/verifier_modeles.py`. Le modèle `qwen3:8b` a été téléchargé mais n'est pas évalué : à quelques jetons par seconde sur cette machine, l'évaluation aurait dépassé toute durée raisonnable. Le modèle de 4 milliards de paramètres n'est évalué que sur un chiffré.

4.4 Le protocole réduit pour les modèles

La machine d'évaluation, décrite dans `results/ENVIRONMENT.json`, est un ordinateur à processeur Intel de 12 cœurs logiques et 15,4 Go de mémoire, sans carte graphique exploitable. Les modèles y sont lents : un chiffré de 300 lettres demande de une à plus de quatre minutes. Mon objectif est de montrer que le concept fonctionne, pas de mesurer des taux précis ; j'ai donc réduit les évaluations avec modèle à quelques minutes de calcul par configuration. Le très petit modèle est évalué sur trois chiffrés de 300 lettres (un César, un Vigenère, une substitution simple), trois fois chacun. Le modèle de 4 milliards de paramètres est évalué sur un seul chiffré, une seule fois. L'expérience d'injection porte sur deux chiffrés. La référence sans modèle est évaluée sur les 720 chiffrés et aussi sur les mêmes chiffrés que chaque configuration avec modèle, ce qui rend la comparaison équitable. Aucun pourcentage obtenu avec un modèle n'a de valeur statistique. Les limites de ce protocole sont rappelées dans la discussion.

4.5 Intégrité de l'évaluation

Les mesures brutes sont écrites au fil de l'eau dans `results/raw/*.jsonl` ; `metriques.py` les agrège en `metrics.json` sans valeur saisie à la main, et ce rapport lit ses nombres dans ce fichier. Les résultats défavorables ne sont ni retirés ni retouchés. Lorsqu'une exécution a été perturbée, par exemple par une mise en veille de l'ordinateur qui fausse un temps de calcul, le fait est signalé à la section concernée plutôt que corrigé en silence.

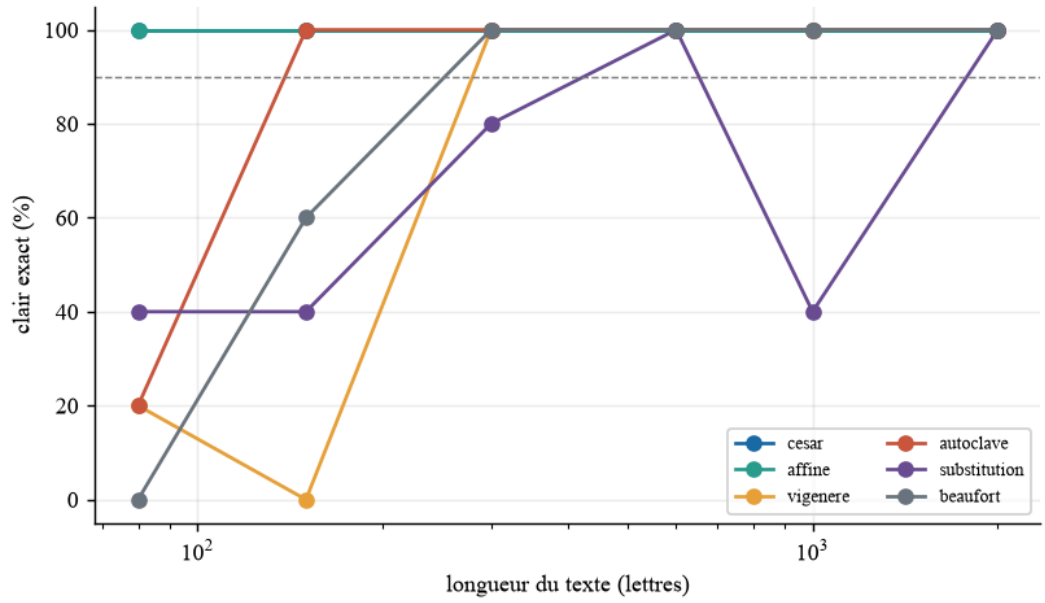
5. Résultats

Tous les nombres de cette section sont lus dans `results/metrics.json`. Je commence par ce que le protocole permet d'affirmer. La référence sans modèle a tourné sur les 180 chiffrés de chacune des quatre variantes, ce qui donne des statistiques solides. Les modèles de langage, eux, n'ont pu être évalués que sur un échantillon minuscule, parce que la machine n'a pas de carte graphique. Ces essais montrent que la chaîne fonctionne de bout en bout avec un vrai modèle ; ils ne permettent pas de comparer des taux de réussite.

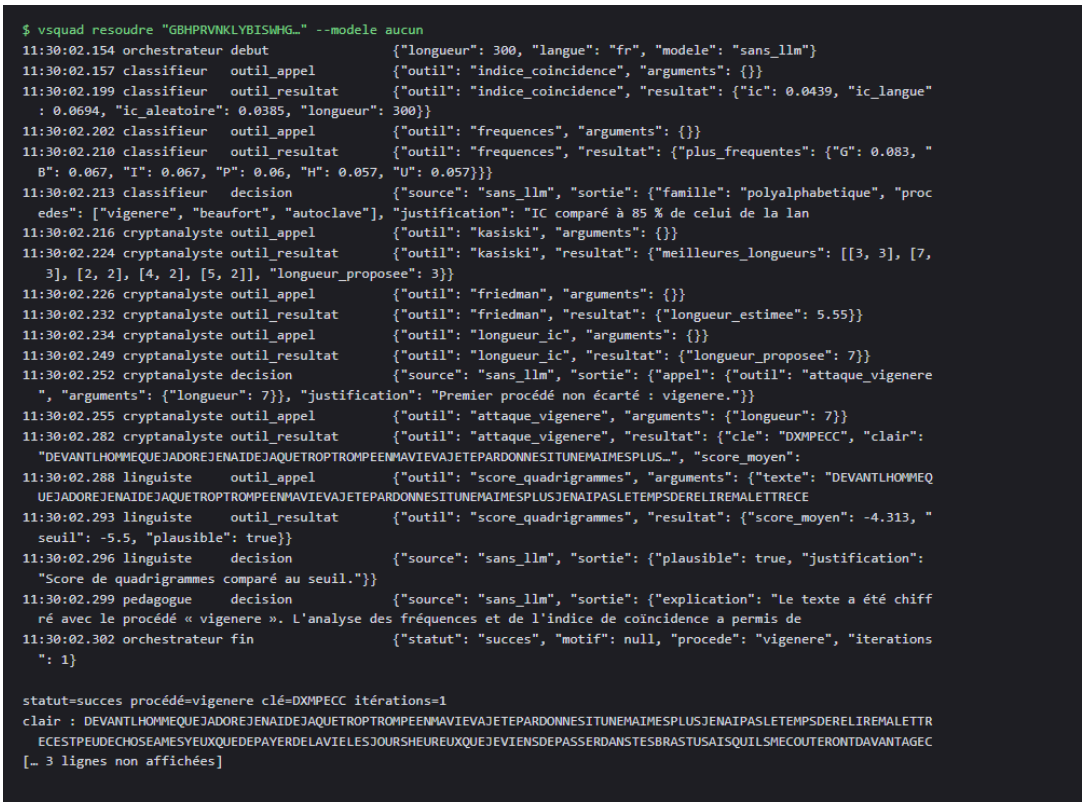
5.1 La référence sans modèle

Sur la variante de base, le système retrouve le clair exact pour 83,3 % des 180 chiffrés, la famille de chiffrement est correctement reconnue dans 98,9 % des cas, et il faut en moyenne 2,1 itérations de l'orchestrateur et 3,8 secondes par chiffré. Il y a 9 échecs, aucun abandon.

Les résultats sont très inégaux selon le procédé. César et le chiffrement affine sont retrouvés dans 100,0 % et 100,0 % des cas : l'espace de clés est si petit que l'on peut les énumérer, et le score de quadrigrammes tranche sans ambiguïté. L'autoclave atteint 86,7 %, Beaufort 76,7 %, Vigenère 70,0 % et la substitution simple 66,7 %. Ces trois derniers dépendent fortement de la longueur du texte : le texte chiffré doit contenir assez de lettres par colonne de la clé pour que les statistiques par colonne soient fiables. La longueur à partir de laquelle toutes les tranches dépassent 90 % de réussite est de 300 lettres pour Vigenère, 300 pour Beaufort, 150 pour l'autoclave, et 2000 pour la substitution simple, qui ne franchit ce seuil qu'avec les textes les plus longs.



Voici la résolution d'un chiffré de Vigenère de 300 lettres sans modèle.



```
$ vsquad resoudre "GBHPRVNKLYBISWHG…" --modele aucun
11:30:02.154 orchestrateur debut               {"longueur": 300, "langue": "fr", "modele": "sans_llm"}
11:30:02.157 classifieur   outil_appel         {"outil": "indice_coincidence", "arguments": {}}
11:30:02.199 classifieur   outil_resultat      {"outil": "indice_coincidence", "resultat": {"ic": 0.0439, "ic_langue"
  : 0.0694, "ic_aleatoire": 0.0385, "longueur": 300}}
11:30:02.202 classifieur   outil_appel         {"outil": "frequences", "arguments": {}}
11:30:02.210 classifieur   outil_resultat      {"outil": "frequences", "resultat": {"plus_frequentes": {"G": 0.083, "
  B": 0.067, "I": 0.067, "P": 0.06, "H": 0.057, "U": 0.057}}}
11:30:02.213 classifieur   decision            {"source": "sans_llm", "sortie": {"famille": "polyalphabetique", "proc
  edes": ["vigenere", "beaufort", "autoclave"], "justification": "IC comparé à 85 % de celui de la lan
11:30:02.216 cryptanalyste outil_appel         {"outil": "kasiski", "arguments": {}}
11:30:02.224 cryptanalyste outil_resultat      {"outil": "kasiski", "resultat": {"meilleures_longueurs": [[3, 3], [7,
   3], [2, 2], [4, 2], [5, 2]], "longueur_proposee": 3}}
11:30:02.226 cryptanalyste outil_appel         {"outil": "friedman", "arguments": {}}
11:30:02.232 cryptanalyste outil_resultat      {"outil": "friedman", "resultat": {"longueur_estimee": 5.55}}
11:30:02.234 cryptanalyste outil_appel         {"outil": "longueur_ic", "arguments": {}}
11:30:02.249 cryptanalyste outil_resultat      {"outil": "longueur_ic", "resultat": {"longueur_proposee": 7}}
11:30:02.252 cryptanalyste decision            {"source": "sans_llm", "sortie": {"appel": {"outil": "attaque_vigenere
  ", "arguments": {"longueur": 7}}, "justification": "Premier procédé non écarté : vigenere."}}
11:30:02.255 cryptanalyste outil_appel         {"outil": "attaque_vigenere", "arguments": {"longueur": 7}}
11:30:02.282 cryptanalyste outil_resultat      {"outil": "attaque_vigenere", "resultat": {"cle": "DXMPECC", "clair":
  "DEVANTLHOMMEQUEJADOREJENAIDEJAQUETROPTROMPEENMAVIEVAJETEPARDONNESITUNEMAIMESPLUS…", "score_moyen":
11:30:02.288 linguiste     outil_appel         {"outil": "score_quadrigrammes", "arguments": {"texte": "DEVANTLHOMMEQ
  UEJADOREJENAIDEJAQUETROPTROMPEENMAVIEVAJETEPARDONNESITUNEMAIMESPLUSJENAIPASLETEMPSDERELIREMALETTRECE
11:30:02.293 linguiste     outil_resultat      {"outil": "score_quadrigrammes", "resultat": {"score_moyen": -4.313, "
  seuil": -5.5, "plausible": true}}
11:30:02.296 linguiste     decision            {"source": "sans_llm", "sortie": {"plausible": true, "justification":
  "Score de quadrigrammes comparé au seuil."}}
11:30:02.299 pedagogue     decision            {"source": "sans_llm", "sortie": {"explication": "Le texte a été chiff
  ré avec le procédé « vigenere ». L'analyse des fréquences et de l'indice de coïncidence a permis de
11:30:02.302 orchestrateur fin                 {"statut": "succes", "motif": null, "procede": "vigenere", "iterations
  ": 1}

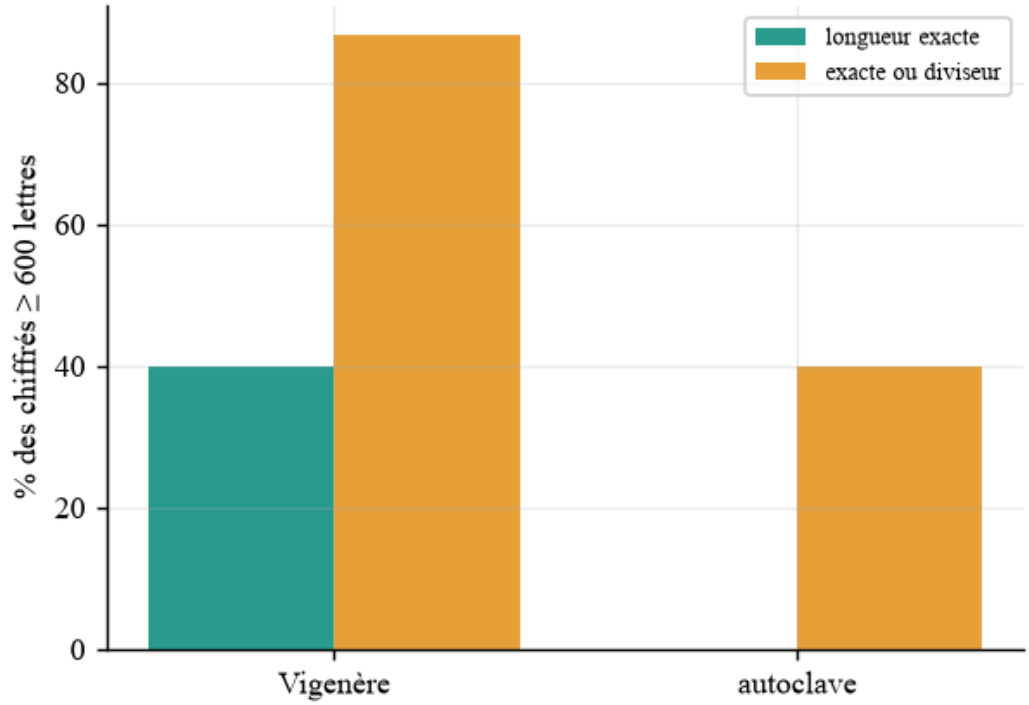
statut=succes procédé=vigenere clé=DXMPECC itérations=1
clair : DEVANTLHOMMEQUEJADOREJENAIDEJAQUETROPTROMPEENMAVIEVAJETEPARDONNESITUNEMAIMESPLUSJENAIPASLETEMPSDERELIREMALETTR
  ECESTPEUDECHOSEAMESYEUXQUEDEPAYERDELAVIELESJOURSHEUREUXQUEJEVIENSDEPASSERDANSTESBRASTUSAISQUILSMECOUTERONTDAVANTAGEC
[… 3 lignes non affichées]
```

5.2 Robustesse aux variantes

La variante sans espaces ni ponctuation donne exactement les mêmes résultats que la base, ce qui est attendu puisque le système normalise le texte avant toute analyse. Ce n'est donc pas une mesure indépendante, et je ne la présente pas comme telle. Les deux autres variantes sont plus informatives. Avec 5 % de fautes de frappe dans le chiffré, le taux de clairs exacts tombe à 77,8 %, et la chute est surtout due à la substitution simple (30,0 %) : une seule lettre fautive modifie le texte clair, et les clairs « exacts » exigent une égalité stricte. Sur le texte en anglais, alors que le modèle de langue du système est français, le taux de clairs exacts reste à 85,0 %, tandis que les attaques les plus fines en souffrent, par exemple Beaufort à 66,7 %.

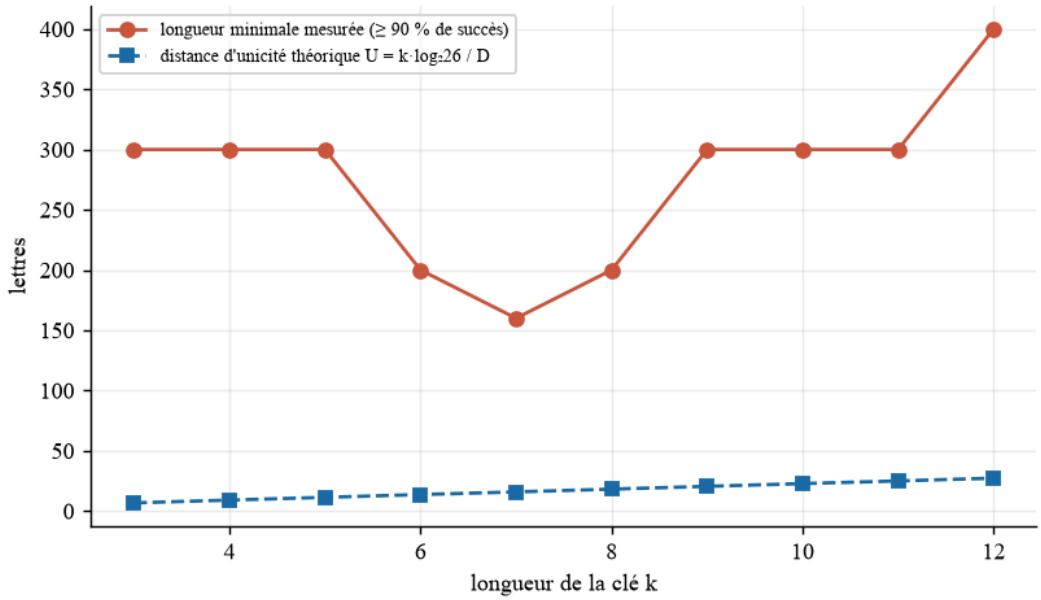
5.3 Kasiski face à l'autoclave

Un des objets d'étude du projet est la limite de la méthode de Kasiski. Pour un Vigenère, elle retrouve la longueur exacte de la clé dans 40,0 % des 15 cas, et la longueur exacte ou un diviseur dans 86,7 %. Pour un autoclave, où la clé se prolonge avec le clair et ne se répète donc pas, elle ne retrouve la longueur exacte dans aucun cas (0,0 %) et un diviseur dans 40,0 % des cas, ce qui est de l'ordre du hasard sur de petites longueurs. C'est cohérent avec la théorie : l'autoclave détruit les répétitions que Kasiski exploite.



5.4 Distance d'unicité

Je mesure la redondance de la langue française par un modèle d'ordre 3 appris sur 3673932 lettres : l'entropie conditionnelle est de 2,63 bits par lettre, soit une redondance D de 2,07 bits par lettre. Pour une clé de Vigenère de longueur 6, la distance d'unicité théorique est de 13,7 lettres et celle de longueur 12 est de 27,3. Expérimentalement, la longueur à partir de laquelle l'attaque retrouve au moins 90 % des clairs est de 200 lettres pour une clé de 6 et de 400 pour une clé de 12. La mesure expérimentale est nettement plus élevée que la théorie, ce qui est normal : la distance d'unicité est le seuil au-dessous duquel plusieurs clés produisent un clair plausible, alors que l'attaque doit en plus trouver la bonne clé avec une méthode statistique imparfaite. Il faut aussi reconnaître que l'entropie d'ordre 3 surestime l'entropie réelle de la langue, si bien que D est sous-estimée et que la distance d'unicité calculée est surestimée.

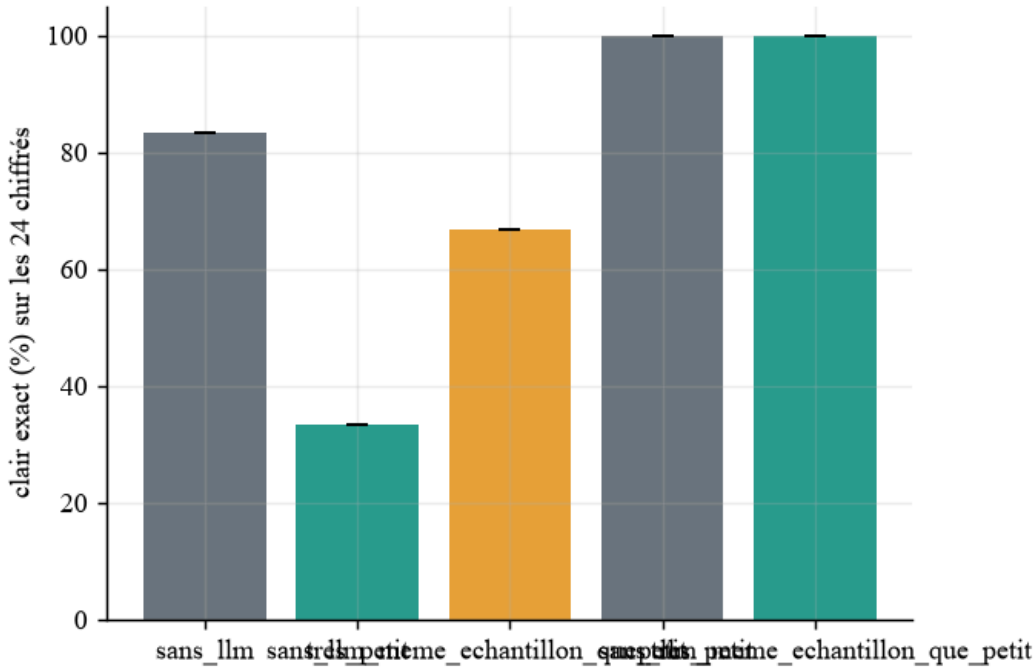


5.5 Ablation : que font les modèles ?

L'ablation compare trois configurations avec la même chaîne d'orchestration : aucun modèle (a), `qwen3:0.6b` (b) et `qwen3:4b` (c). Le protocole a dû être fortement réduit. Le tableau suivant donne les nombres de chiffrés effectivement évalués pour chaque configuration, car sans eux les pourcentages seraient trompeurs.

Configuration	Chiffrés distincts	Répétitions	Clairs exacts	Durée moyenne par chiffré (s)	Appels au modèle par chiffré
(a) sans modèle, mêmes chiffrés que (b)	3	1	66,7 %	non mesurée ici	0
(b) `qwen3:0.6b`	3	3	33,3 ± 0,0 %	115,6	9,3
(a') sans modèle, même chiffré que (c)	1	1	100,0 %	non mesurée ici	0
(c) `qwen3:4b`	1	1	100,0 %	248,7	4,0

Les chiffrés choisis ont 300 lettres : un César, un Vigenère et une substitution simple pour (b), le César seul pour (c). Le modèle de 0,6 milliard de paramètres retrouve le César dans les trois répétitions, et échoue sur le Vigenère et sur la substitution. Sans modèle, le même orchestrateur retrouve le César et le Vigenère et échoue sur la substitution. Sur ces trois chiffrés, le petit modèle fait donc moins bien que la référence : il perd le Vigenère. Il a produit 4 sorties invalides au total sur les neuf exécutions, et l'agent est revenu 2 fois à sa décision déterministe. Le modèle de 4 milliards de paramètres retrouve le César, comme la référence, mais en 249 secondes. Je ne tire aucune conclusion sur son efficacité générale à partir d'un seul chiffré.



6. Modèle de menace du système lui-même

Le modèle de langage est une entrée non fiable, et le texte chiffré lui-même peut contenir une consigne. Le tableau STRIDE complet figure dans `docs/MENACES.md`. Trois expériences en évaluent les contre-mesures.

6.1 Données d'outil empoisonnées

Je simule des outils de cryptanalyse (le χ^2 de César et l'attaque de Vigenère) qui annoncent un clair français fluide mais sans rapport avec le chiffré, avec le score de plausibilité le plus favorable. Le contrôle de cohérence de l'orchestrateur rechiffre le clair annoncé avec la clé annoncée et vérifie que l'on retombe exactement sur le chiffré. Sur 10 chiffrés (cinq César et cinq Vigenère), le système sans ce contrôle accepte 10 faux succès sur 10, c'est-à-dire tous. Avec le contrôle, il en accepte 0. Les cinq chiffrés de César sont alors tout de même résolus correctement, par l'énumération du chiffrement affine, dont l'outil n'était pas empoisonné ; les cinq chiffrés de Vigenère se terminent par un échec déclaré, ce qui est le résultat honnête quand le seul outil disponible ment. Le contrôle de cohérence est donc une contre-mesure efficace contre cette menace, et peu coûteuse puisqu'il ne repose que sur un rechiffrement. Sa limite est qu'il suppose que l'outil de chiffrement lui-même n'est pas corrompu.

6.2 Outil halluciné

Je simule un agent qui appelle un outil inexistant ou hors de sa liste. L'exécuteur refuse ces appels (20 refus sur 10 chiffrés), le système se replie 30 fois sur sa décision déterministe, et les 10 chiffrés sont tout de même résolus. L'hallucination coûte donc des appels perdus, mais pas de résultat.

6.3 Consigne cachée dans le clair

Je place une consigne dans le texte clair, avec un mot-témoin inoffensif (le canari) que le modèle devrait répéter s'il obéissait. Un masquage des extraits de clair, que l'orchestrateur peut activer, empêche le modèle de les lire.

6.4 Résultat de l'expérience d'injection

Avec `qwen3:0.6b`, je lance deux chiffrés (un César et un Vigenère), une fois avec et une fois sans masquage des extraits de clair, soit 2 exécutions dans chaque cas. Le mot-témoin n'apparaît dans aucune sortie d'agent ni dans aucune explication, avec ou sans masquage (0,0 % et 0,0 %). Je ne peux pas en conclure que le masquage est utile : avec deux chiffrés, l'expérience ne distingue pas un modèle qui ignore la consigne d'un modèle qui n'a pas lu le clair. Elle montre que la mesure est en place et fonctionne ; il faudrait un échantillon bien plus grand, et un modèle plus obéissant, pour que le résultat soit informatif.

7. Discussion et limites

7.1 Ce que les agents apportent réellement

La question posée par le sujet est de savoir si les agents apportent quelque chose par rapport aux outils seuls. Mes mesures permettent une réponse partielle et prudente. Les outils déterministes font la cryptanalyse, et le code décide du succès : ni le classement de familles ni la plausibilité ne dépendent du modèle. Sans modèle, le système obtient déjà 83,3 % de clairs exacts. L'apport des agents ne peut donc se situer que dans le choix des outils et dans l'explication. Sur l'échantillon que j'ai pu évaluer, aucun résultat ne montre que les agents améliorent la cryptanalyse. Le très petit modèle a fait moins bien que la référence sur trois chiffrés, et le modèle de 4 milliards de paramètres a fait aussi bien sur un seul chiffré, avec un temps de calcul des centaines de fois supérieur à celui des outils seuls. Ce que les modèles apportent de manière plausible est ailleurs : le Pédagogue rédige des explications, avec une note moyenne de la grille automatique de 5,0 sur 10 pour le petit modèle (une explication évaluée), ce qui est trop peu pour conclure. Mon résultat est donc négatif ou nul à l'échelle où je l'ai mesuré, et je le publie tel quel.

7.2 Limites

La limite principale est le faible effectif des évaluations avec modèle. Je n'ai ni les ressources de calcul ni le temps pour évaluer les 180 chiffrés avec chaque modèle, et un échantillon de quelques chiffrés ne donne pas de taux : il montre que la chaîne fonctionne. Les trois répétitions mesurent la

variabilité d'un même modèle, pas la diversité des chiffrés. Un deuxième point est la variante sans espaces, qui n'est pas une mesure indépendante. Troisièmement, une mise en veille de l'ordinateur pendant une exécution antérieure a gonflé la durée d'un chiffré (61 484 s) ; cette exécution a été abandonnée et archivée dans `results/raw/archive_essai_interrompu`, et aucune durée n'en est tirée. Enfin, les modèles évalués sont petits, et un modèle plus capable ou une consigne cachée plus soignée pourraient réussir là où ils échouent.

7.3 Suite possible

Évaluer un plus grand échantillon sur une machine avec GPU, mesurer l'effet de modèles plus grands, et comparer des stratégies de choix d'outils apprises plutôt que fixées.

8. Reproductibilité

L'installation tient en quelques commandes décrites dans le `README.md` : cloner le dépôt, `uv sync`, `make test`, puis `make reproduce-small`. Cette dernière commande rejoue une version courte avec `qwen3:0.6b` seulement, en quelques minutes, et c'est celle que lance l'intégration continue. La commande `make reproduce` rejoue le protocole de ce rapport. Les modèles sont identifiés par leur empreinte dans `config/experiment.yaml` et l'environnement mesuré est enregistré dans `results/ENVIRONMENT.json`. Les traces JSONL de chaque exécution sont dans `traces/` et se rejouent avec `vsquad rejouer --verifier`. Une réexécution redonne les mêmes résultats pour la référence sans modèle, qui est déterministe, mais seulement des résultats proches pour les modèles de langage, dont la sortie dépend du matériel et de la version d'Ollama.

```
$ pytest -q --cov
.....................................................................          [100%]
============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.10-final-0 _____________

Name                                      Stmts   Miss  Cover
-------------------------------------------------------------
src\vigenere_squad\outils\__init__.py         0      0   100%
src\vigenere_squad\outils\attaques.py        84      0   100%
src\vigenere_squad\outils\chiffres.py        69      0   100%
src\vigenere_squad\outils\coherence.py       24      4    83%
src\vigenere_squad\outils\langue.py          37     10    73%
src\vigenere_squad\outils\registre.py        57      7    88%
src\vigenere_squad\outils\stats.py           59      0   100%
-------------------------------------------------------------
TOTAL                                       330     21    94%
62 passed in 13.29s
```

9. Conclusion

J'ai construit un système de quatre agents qui casse six chiffrements classiques en laissant à du code déterministe la cryptanalyse et la décision de succès. La référence sans modèle donne 83,3 % de clairs exacts sur 180 chiffrés, avec des résultats très inégaux selon le procédé et la longueur, et

une limite nette de la méthode de Kasiski sur l'autoclave. Les contre-mesures contre les données d'outil empoisonnées et les outils hallucinés fonctionnent, et sont mesurées. L'évaluation avec modèles est trop réduite pour conclure sur leur apport, ce que je dis plutôt que de le masquer.

Annexe A. Intégrité et usage de l'IA

Les nombres du rapport sont lus dans `results/metrics.json`. Le code et les premières versions des textes ont été écrits par l'assistant Claude Code à partir de mes instructions ; ce que j'ai moi-même fait, relu et compris est décrit dans `AI_USAGE.md`. Aucun secret ne figure dans le dépôt, les prompts ou les traces. Les résultats défavorables sont publiés.